# Worktual AI Website Builder

Complete AI Agent, Orchestration, Tool Calling, ADK, LangChain, A2A, and Website Generation Flow

This document explains the complete runtime flow of the AI-native website builder: how user prompts enter the system, how AI agents and dynamic agents operate, how Gemini tool calling is guarded by Python, how Google ADK/LangChain/A2A projections are used, and how generated websites are validated, previewed, persisted, and reused.

### End-to-End Flow

User prompt -> Frontend workspace -> FastAPI generation endpoint

Generation pipeline -> Gemini routing -> Orchestrator -> Real agent/tool loop

Backend tools -> Validation -> Staged preview -> Visual QA -> WRITE_PROJECT_FILES

Local sync + memory persistence -> UI progress + preview

## 1. Executive Summary

Worktual AI Website Builder is an AI-native website generation and update platform. It accepts a user prompt, routes the request, prepares project context, coordinates AI agents and backend tools, generates or patches React/Vite website files, validates the artifact, builds a preview, runs visual QA, persists files, and records runtime memory for reuse.

- Website generation creates a complete first project from user intent and context.
- Website update reads existing files and applies deterministic or scoped model patches.
- Gemini reasons and proposes outputs; Python validates, executes tools, builds previews, and writes files.

## 2. User Query Entry

The user starts in the React workspace (`src/main.jsx`). Prompts are sent to FastAPI endpoints in `backend/main.py` and `backend/api/generation.py`. The backend loads the project, permissions, current files, selected model, local folder state, telemetry, and runtime context before invoking the AI generation pipeline.

**Prompt Entry**

Chat input / workspace action
-> API generation endpoint
-> project + permission load
-> GeminiProvider setup
-> persistent agent_run created

## 3. Main Agents

| Agent / Stage | Responsibility |
|---|---|
| Intent Router | Classifies greeting, clarification, website generation, or website update. |
| Prompt Analyst | Turns prompt, existing files, memory, and domain context into a structured brief. |
| Domain Research Agent | Adds domain assumptions, audience, content requirements, interactions, and sample data. |
| UX/Layout Agent | Plans structure, pages, sections, interactions, and responsive behavior. |
| Accessibility Agent | Reviews semantic HTML, contrast, keyboard flow, ARIA, and mobile fit. |
| Dynamic Task Decomposer | Breaks complex work into capability tasks. |
| Dynamic Workflow Planner | Builds guarded workflow plans and parallel groups. |
| Dynamic Specialists | Execute bounded domain/capability work and propose candidate changes. |
| Code Generator | Generates the strict website artifact JSON and files. |
| Scoped Update Agent | Produces SEARCH/REPLACE edits for existing code updates. |
| Repair Agent | Repairs invalid artifacts, failed builds, and QA failures when safe. |
| Memory Agent | Loads and persists project memory and reusable dynamic agents. |

## 4. Orchestrator and Runtime Loop

The orchestrator prepares a response through staged sections: routing, multi-agent system metadata, Gemini tool-calling setup, Google ADK usage, orchestration flow, agent-to-agent communication, and proactive thinking.

**Runtime Loop**

READ_PROJECT_FILES -> LOAD_PROJECT_MEMORY

RUN_PROMPT_ANALYST or RUN_UPDATE_ANALYST

RUN_DYNAMIC_AGENT_PLANNER -> RUN_DYNAMIC_SPECIALISTS

RUN_PLANNER -> RUN_UX_REVIEW_AGENT -> RUN_ACCESSIBILITY_AGENT

RUN_CODE_AGENT or RUN_SCOPED_UPDATE_AGENT

VALIDATE_PROJECT_ARTIFACT -> BUILD_STAGED_PROJECT_PREVIEW

RUN_PREVIEW_VISUAL_QA -> WRITE_PROJECT_FILES -> PERSIST_PROJECT_MEMORY

For non-generation turns such as greetings or missing details, the orchestrator returns a conversation response and does not run the artifact workflow.

## 5. Dynamic Agents

Dynamic agents are created, assigned, reused, promoted, disabled, and persisted through `backend/llm/dynamic_agenting`. They let the builder specialize for capabilities such as component UI, CRM pipeline logic, RBAC, e-commerce catalogs, support flows, and domain content.

- Creation: the task decomposer emits capability tasks; the registry looks for reusable agents; if none exists and policy allows, a model-generated reusable definition is sanitized and registered.
- Reuse: user-scoped dynamic agents are hydrated from storage, scored by capability/domain/success metrics/usage/lifecycle, and assigned to future tasks.
- Safety: no direct file writes, bounded tools, candidate change limits, rejection of project-specific prompts, and Python-only tool execution.
- Promotion: repeated successful specialists can move from experimental to reusable after meeting configured success thresholds.

## 6. Gemini Tool Calling

Gemini is used in control and artifact roles. Control handles routing, planning, reviews, and update analysis. Artifact handles website artifact creation and scoped patch generation. Native Gemini function calling is guarded by Python.

- Python converts backend tools into Gemini function declarations.
- Gemini may request a tool call.
- Python validates and executes the tool.
- Tool result is sent back as a function response.
- The loop continues until final output or max-step exhaustion.
- All model calls, tool calls, status, errors, and usage are logged.

## 7. Google ADK, LangChain, and A2A

| Layer | How it is used |
| --- | --- |
| Google ADK | Runtime metadata/projection layer. Builds ADK agent plan, tool specs, sessions, events, runner metadata, and validation when google-adk is installed. |
| LangChain / LangGraph | Compatibility projection. Converts real runtime steps into graph nodes/edges, messages, thread IDs, and optional LangGraph app when dependencies exist. |
| A2A Communication | Builds canonical agent handoff transcripts, acknowledgements, channel routing, confidence scores, and validation from runtime steps. |

These layers make the runtime explainable and portable. They do not replace the Python source of truth for validation, preview building, and file writes.

## 8. Website Artifact Contract

Generated websites follow a strict JSON contract containing title, headline, calls to action, theme, design tokens, component manifest, SEO metadata, compliance checks, sections, files, and implementation notes.

- `src/App.jsx` should be a thin composition shell.
- `src/components/*` contains reusable atomic components.
- `src/pages/*` contains route-backed modules when the site has real pages.
- `src/data/*` contains realistic domain data.
- `src/theme/tokens.js` contains dynamic or user-provided design tokens.
- `src/seo/schema.js` or equivalent contains JSON-LD structured data.

## 9. Website Update Flow

Updates are context-preserving. The backend sends existing files to Gemini, selects a deterministic or scoped strategy, and requires explicit SEARCH/REPLACE blocks for scoped edits. Python verifies each search block against the actual current file before merging.

**Update Reliability**

Existing files -> update analysis -> candidate files

Scoped SEARCH/REPLACE -> parser/validator -> candidate integration

Staged build -> visual QA -> write files

## 10. Backend Tools and Quality Gates

| Tool / Gate | Purpose |
| --- | --- |
| READ_PROJECT_FILES | Reads current supported files from backend store or linked local workspace. |
| LOAD_PROJECT_MEMORY | Loads persisted project memory and dynamic agent registry snapshots. |
| VALIDATE_PROJECT_ARTIFACT | Validates artifact fields, paths, theme, required files, and React safety. |
| BUILD_STAGED_PROJECT_PREVIEW | Builds preview from candidate files before commit. |
| RUN_PREVIEW_VISUAL_QA | Checks staged preview integrity. |
| WRITE_PROJECT_FILES | Commits validated files to backend storage and linked local folder. |
| PERSIST_PROJECT_MEMORY | Stores final memory, dynamic registry data, and runtime summary. |

## 11. Local Workspace Behavior

The builder supports backend-only projects, browser-selected folders, browser-uploaded folders, server-local folders, and empty folders. Empty folders are valid because the user may want AI to create the first website files there. Existing file imports are validated so partial or incorrect project roots can be warned or rejected.

## 12. Source Map

| Area | Files |
| --- | --- |
| Frontend workspace | `src/main.jsx` |
| API endpoints | `backend/main.py`, `backend/api/*` |
| Generation pipeline | `backend/api/generation.py`, `backend/llm/generator/*` |
| Prompts/contracts | `backend/llm/prompting/*` |
| Gemini client/tool calling | `backend/llm/gemini_client/*`, `backend/llm/gemini_tool_calling/*` |
| Orchestrator | `backend/llm/orchestration/*`, `backend/llm/orchestrator.py` |
| Runtime loop | `backend/llm/agent_runtime_loop.py`, `backend/llm/agent_runtime/*` |
| Dynamic agents | `backend/llm/dynamic_agenting/*` |
| Backend tools | `backend/agentic/tools/*` |
| ADK/LangChain/A2A | `backend/llm/google_adk_runtime/*`, `backend/llm/langchain_runtime_impl/*`, `backend/llm/a2a/*` |
| Validation/local sync | `backend/llm/artifacts/*`, `backend/local_workspace/*` |

## 13. Operational Summary

The platform is safest when Gemini is treated as the reasoning and generation engine while Python remains the execution authority. Gemini routes, analyzes, plans, and proposes. Python validates, executes tools, builds previews, writes files, synchronizes local folders, persists memory, and produces an explainable runtime trail through ADK, LangChain, and A2A projections.